

Downwrite roadmap

Candidate features that Downwrite does not have yet, with the reasoning for (and against) each. Last reviewed 2026-10-10.

How to use this file

- Every item has a permanent reference of the form `DW-NNN`, for example "create the intent for roadmap item DW-008". References are never renumbered or reused, even if an item is reordered, dropped or shipped.
- The order of the items in this file is the current priority, highest first. Re-ordering the sections changes the priority; it does not change the references.
- Status values: **Proposed** (nobody has started), **Intent drafted** (an intent exists in `docs/changes/`), **In progress**, **Released** (say which tag), **Dropped** (say why). When an item moves on, cite its `DW-NNN` in the change's `intent.md` so the two can be traced.
- When a roadmap item is fully released, add its row to `FEATURE-LIST.md` as `CLAUDE.md` describes, and set its status here to Released.
- "What the code has today" was checked against the source on the review date above. It is evidence for the "why", not a design.

Summary

Ref	Item	Tier	Status
DW-001	Export and print	1	In progress (merged, not yet released)
DW-002	Paste and drop images	1	Proposed
DW-003	Syntax highlighting in code blocks	1	Proposed
DW-004	Typeset math	1	Proposed
DW-005	Folders, quick open and linking between notes	1	Proposed
DW-006	GitHub-style callouts	2	Proposed
DW-007	Focus and typewriter mode	2	Proposed
DW-008	Auto-pairing while typing	2	Proposed
DW-009	Localisation	2	Proposed
DW-010	System integration (Shortcuts, Services, AppleScript)	2	Proposed
DW-011	Accessibility audit	Unverified risk	Proposed

Tier 1 is the set of gaps most likely to stop someone using Downwrite every day. Tier 2 is worth doing but would not stop anyone adopting it. The last item is an audit of something that has not been tested yet.

Running two items in parallel

Two agents can work on two items at once, each in its own git worktree and branch, provided the items do not edit the same files. This section records which pairs look safe. It is a judgement from the "What the code has today" notes below, not from a design, so check it against the file list in each item's `plan.md` before relying on it. Pairs are written with the permanent `DW-NNN` references, so they stay valid when the items are re-ordered.

Safe to run together (different parts of the code):

Pair	Why they should not collide
DW-002 + DW-006	Paste and drop live in <code>EditorTextView</code> and <code>ImageLoader</code> . Callouts live in <code>MarkdownAnalyzer</code> , <code>MarkdownStyler</code> and <code>DWLayoutManager</code> .
DW-006 + DW-010	Callouts are inside the editor. Services and Shortcuts are <code>Info.plist</code> entries and new App Intents files.
DW-002 + DW-010	Same split: the paste override against <code>Info.plist</code> and new intent files.
DW-002 + DW-004	Image paste is in <code>EditorTextView</code> . Block math uses the card machinery (<code>DiagramOverlay</code> , <code>PreviewBlock</code>) and a bundled library.
DW-001 + DW-008	Export is new code plus File menu commands. Auto-pairing is in <code>EditorTextView.insertText</code> and <code>DownwriteCore</code> .

Avoid running together (same files or the same decision):

Pair	Why
DW-003 + DW-004	Both turn on the bundled-library question, so both edit <code>Package.swift</code> or <code>build-app.sh</code> , <code>THIRD-PARTY-NOTICES.md</code> and <code>ThirdParty/</code> , and both change <code>MarkdownAnalyzer</code> . Settle the approach once, then do one after the other.

DW-003 + DW-006, DW-003 + DW-007, DW-006 + DW-007	All three work in the styler, the analyzer and <code>DWLayoutManager</code> , which is where merge conflicts and restyling bugs are most likely.
DW-009 + any item that adds or changes UI text	Localisation touches nearly every file in the app target. Do it alone, or last.

Order matters (a dependency rather than a conflict):

- DW-001 can start alongside DW-002, DW-003 or DW-004, but its export of pasted images, highlighting and math is only complete once those ship.
- DW-005 needs the direction decision before any intent. After that, each slice (Open Quickly, folder sidebar, search, wikilinks) is a separate candidate for a pair.
- DW-011 is read-only until its findings become fixes, so it can run beside anything. Running it before DW-006 or DW-007 lets the new custom-drawn elements follow what it finds.

Tier 1: most likely to stop daily use

DW-001 — Export and print

- **Status:** In progress. Print..., Export as PDF... and Export as HTML... merged in PR #17 on 2026-10-10 (docs/changes/2026-10-10-export-and-print/); it becomes Released, with its tag, once a release ships it.
- **Why add it:** A Markdown document is usually written so that someone else can read it, and today the only way out of Downwrite is to hand over the `.md` file itself. There is no PDF, HTML or Word export and no Print command, which every Mac document app is expected to have. People hit this in their first week and return to another tool for the final step.
- **Why not, or what to watch:** Word (`.docx`) is a different size of job from PDF and HTML because there is no library for it and it would mean a new dependency or hand-written OOXML, so a sensible first slice is Print, Export as PDF and Export as HTML, with Word deferred. Export has to make decisions the editor never has to: ignore folds and the source view, reproduce tables, Mermaid diagrams, checkboxes and HTML blocks, and use a print-friendly light style even when the window is in a dark theme.
- **What the code has today:** No print or export code in `Sources/` , and the File menu has Share only (from the CI menu dump in `docs/ci/menus.txt` , which may be older than the code). Useful building blocks exist: `HTMLSupport.page` already builds a locked-down HTML page for rendered HTML blocks, and `MermaidService` produces SVG.
- **Known gaps in the first slice:** Print and PDF are paginated by WebKit's print engine, which ignores `break-after: avoid` and never repeats a table's `<thead>` on later pages (found by the spike in `docs/changes/2026-10-10-export-and-print/plan.md`). So a table that runs over several pages shows its header row only where it starts, and a heading is kept with the block after it only when both fit on one page (a heading above a block taller than a page may end a page). Fixing either would take Downwrite's own page layout (measuring the laid-out page and moving breaks), which is a separate piece of work. The three File commands also do not grey out while a preparation is under way or with only Settings frontmost (R2; the controller still refuses a second command, see the plan's *Open* section).
- **Related:** DW-002 (exports should embed pasted images), DW-003 and DW-004 (exports should carry highlighting and math once they exist).

DW-002 — Paste and drop images

- **Status:** Proposed
- **Why add it:** Images already preview inline, but getting one into a document is manual: save the file somewhere, then type its path. Pasting a screenshot or dragging a photo in is how people actually add images, and users of other Markdown editors expect it. Without it, the image preview feature is only half used.
- **Why not, or what to watch:** The image has to be saved somewhere, normally in an assets folder next to the document, and an untitled, unsaved document has no folder. That needs a decision (ask to save first, or hold the image in a temporary place and move it on save). It also needs rules for file naming and collisions, for converting pasteboard formats (TIFF, HEIC) to PNG or JPEG, and for what happens to very large images. The app is not sandboxed, so writing next to the document is possible.
- **What the code has today:** The `paste` override in `EditorTextView` only handles a URL pasted over selected text. There is no image pasteboard handling and no drag destination for files. The preview side exists: `ImageLoader` turns local images into `data:` URIs with an 8 MB cap.
- **Related:** DW-001.

DW-003 — Syntax highlighting in code blocks

- **Status:** Proposed
- **Why add it:** Fenced code is shown in a single monospaced style, so it looks unfinished next to the themed cards and careful typography around it. Anyone writing READMEs, documentation or tutorials expects ````swift` to colour the code. The language is already parsed, and imported VS Code themes already carry `tokenColors` scopes that Downwrite reads and matches, so the colour data for token types partly exists.
- **Why not, or what to watch:** This is where the project rule "keep the app light: no new dependencies without a strong reason" bites. The options are a hand-written tokenizer in `DownwriteCore` for a short list of languages (no dependency and testable on Linux, but limited coverage and ongoing upkeep) or a bundled highlighter such as highlight.js (broad coverage, but it adds size and a licence entry in `THIRD-PARTY-NOTICES.md` and `ThirdParty/`, and its output has to be mapped back onto text attributes because code cards are drawn by TextKit, not a web view). Every built-in palette would need token colours and keep to the legibility floors in `ThemeCatalogTests`.
- **What the code has today:** The fence language is read in `MarkdownAnalyzer` only to recognise `mermaid`. All other code is one monospaced style with no token colours.
- **Related:** DW-004 (both are a bundled-library decision), DW-001.

DW-004 — Typeset math

- **Status:** Proposed
- **Why add it:** `$...$` and `$$...$$` are recognised and styled but shown as raw LaTeX, which the README already lists as a limitation. Anyone writing science, engineering or academic notes sees source instead of equations. A bundled KaTeX would work offline in the same way Mermaid does.
- **Why not, or what to watch:** The two forms are very different in cost. Block math (`$$...$$`) fits the existing card machinery used for Mermaid and HTML (collapsed source, rendered card under it). Inline math is much harder, because the editor draws inline text itself and the same limit already applies to images inside a paragraph (styled, not previewed), so inline math needs a new mechanism such as an inline attachment aligned to the baseline. A sensible slice is block math first. KaTeX also adds JavaScript, CSS and fonts to an app that advertises being small, plus a licence entry. The existing regexes already avoid some false positives such as `$5` and `$10`.
- **What the code has today:** `MarkdownAnalyzer` produces `.math` spans for inline and block math, and nothing renders them.
- **Related:** DW-003 (same bundled-library question), DW-001.

DW-005 — Folders, quick open and linking between notes

- **Status:** Proposed
- **Why add it:** People who keep notes or docs in a folder, such as an Obsidian vault or a `docs/` directory in a repository, move between files constantly. Today each file is its own window or tab, with no folder view, no quick-open, no search across files, and no `[[wikilinks]]` or backlinks. For an app that wants to be the default `.md` editor, this is the gap that sends heavy users to a notes app.
- **Why not, or what to watch:** This is the biggest scope decision on the list, and "do not build it" is a reasonable answer. Downwrite is deliberately a light, one-document-per-window editor, and folder features pull it towards a notes app with indexing, file watching, search and persistent state to maintain, which conflicts with "keep the app light". If it goes ahead it should be sliced so each piece ships alone: Open Quickly over recent files, then a folder sidebar, then search across files, then wikilinks and backlinks. Decide the direction before writing an intent.
- **What the code has today:** The app is built on `DocumentGroup` and has no folder, quick-open, cross-file search or wikilink code. Small foundations exist: relative file links open the target (`EditorCoordinator.open(destination:)`) and `#heading` links scroll to the heading.

Tier 2: worth doing, would not stop adoption

DW-006 — GitHub-style callouts

- **Status:** Proposed
- **Why add it:** `> [!NOTE]`, `> [!TIP]`, `> [!WARNING]` and similar render as coloured callouts on GitHub and in several editors, and documentation written for GitHub uses them heavily. In Downwrite they show as a plain quote with the literal `[!NOTE]` text, so the author sees something different from the reader.
- **Why not, or what to watch:** The cost is low for the value: it is a variant of a block quote, parsed in `DownwriteCore` (so testable on Linux), drawn with a coloured bar and a label, with the marker line hidden like other syntax. It could reasonably move up the list. Decisions needed: which types to support, whether to accept other editors' variants such as foldable `[!note]+` callouts, and how callouts interact with folding. Elsewhere they degrade gracefully to a normal quote.
- **What the code has today:** Block quotes are drawn with a quote bar, and there is no `[!TYPE]` handling.

DW-007 — Focus and typewriter mode

- **Status:** Proposed
- **Why add it:** A common writing aid for long-form text: dim everything except the current line, sentence or paragraph, and keep the caret vertically centred. It suits the "quiet, beautiful to type in" positioning.
- **Why not, or what to watch:** It is not a Markdown capability, so it is a nice-to-have rather than a gap. It adds another display layer that has to cooperate with folding, overlay cards and the source view, and dimming means more attributes in the styler, which is sensitive to performance because lines are restyled as the caret moves. It would follow the source view's pattern: per window and off by default.
- **What the code has today:** Nothing similar.

DW-008 — Auto-pairing while typing

- **Status:** Proposed
- **Why add it:** Many editors close brackets, quotes and backticks as you type them, and Downwrite already does this for code fences (`FenceEditing`). Closing a bracket or backtick yourself is a small interruption to writing flow.
- **Why not, or what to watch:** This is the weakest item in tier 2 and may be better dropped. Auto-pairing is arguably wrong for Markdown: a lone `*` or `_` is common (list markers, `snake_case`), so pairing leaves stray markers, and a pair inserted automatically interacts awkwardly with syntax that hides and reveals around the caret. Smart quotes are deliberately switched off (`isAutomaticQuoteSubstitutionEnabled = false`), which suggests literal typing is the intended feel. The Format shortcuts (⌘B, ⌘E, ⌘K) already wrap a selection or insert an empty pair. If anything is done, limit it to brackets, backticks and quotes, or wrapping a selection when you type a marker.
- **What the code has today:** Format commands wrap or insert pairs, `FenceEditing` closes a fence, and there is no per-character pairing.

DW-009 — Localisation

- **Status:** Proposed
- **Why add it:** Downwrite is English only, with no `Localizable` strings or String Catalog, while Markdown users are worldwide. SwiftUI text literals pick up a String Catalog with little work, so a first set of languages would not be a big job.
- **Why not, or what to watch:** The ongoing cost is higher than the first pass. Every UI string change needs re-translation and native review. Strings built in AppKit code (alerts, the table grid's menus) need `NSLocalizedString` rather than being picked up automatically. The end-to-end self-test checks Settings tabs by their window title, so tests would have to pin the locale. Right-to-left languages would need the custom-drawn bars, checkboxes and table grid checked, which has not been tested. It may be better to wait for evidence of demand.
- **What the code has today:** No localisation setup; all strings are English literals.

DW-010 — System integration (Shortcuts, Services, AppleScript)

- **Status:** Proposed
- **Why add it:** Mac power users automate their writing: Shortcuts actions (App Intents) such as "append to document" or "get word count", a Services entry to start a Downwrite document from selected text in any app, and AppleScript. It makes Downwrite part of existing workflows.
- **Why not, or what to watch:** The audience is small, and every surface is an API to keep stable. App Intents have to work with a `DocumentGroup` document model, which is not trivial. Services alone (an `NSServices` entry in `Info.plist`) is a cheap first slice if any of this is done. It can wait for demand.
- **What the code has today:** No App Intents, Services entries or AppleScript support.

Unverified risk

DW-011 — Accessibility audit

- **Status:** Proposed
- **Why add it:** The task checkboxes, fold chevrons and chips, quote bars and table grid are drawn by `DWLayoutManager` or custom views rather than being native controls, so VoiceOver may not expose them (whether a task is ticked, whether a section is folded). Accessibility is a matter of correctness, not polish, and finding out how bad it is costs little.
- **Why it is last, or what to watch:** It is an audit rather than a feature, and its result is unknown until someone tests with VoiceOver, Voice Control and Full Keyboard Access. It could rank higher, and moving it up costs nothing. The deliverable would be a findings list, each finding then becoming its own item or fix.
- **What the code has today:** Accessibility labels are set in four source files (`Previews.swift`, `EditorTextView.swift`, `TOCSidebar.swift`, `TableGrid.swift`), so there is some coverage; how much, and whether the checkboxes and fold controls are included, is not known. Accessibility identifiers in six files are there for the end-to-end test, not for assistive technology. No VoiceOver testing has been done or recorded.